

4_핵심소스코드_A축

핵심 소스코드 — A축 (개인 GPS 습관 인증)

Booster — AI 습관 형성 서비스 / A축 근거: `backend/src/main/java/com/booster/**` 실제 구현 코드 양식:
[서식2] 제작설계서 PPTX 슬라이드 27·28 (코드 + 설명)

| 핵심 소스코드 (1) — GPS 반경 판정(Haversine 알고리즘)

핵심 적용 기술. 두 위경도 좌표 사이의 지표면 거리(m)를 구해 등록 반경과 비교한다.

```
// shared/gps/GpsVerificationEvaluator.java
@Component
public class GpsVerificationEvaluator {

    private static final double EARTH_RADIUS_METERS = 6_371_000.0;

    public boolean isWithinRadius(double registeredLat, double registeredLng,
                                  int radiusMeters,
                                  double currentLat, double currentLng) {
        double distance = haversineDistance(registeredLat, registeredLng, currentLat, currentLng);
        return distance <= radiusMeters;
    }

    private double haversineDistance(double lat1, double lng1, double lat2, double lng2) {
        double dLat = Math.toRadians(lat2 - lat1);
        double dLng = Math.toRadians(lng2 - lng1);
        double a = Math.sin(dLat / 2) * Math.sin(dLat / 2)
            + Math.cos(Math.toRadians(lat1)) * Math.cos(Math.toRadians(lat2))
            * Math.sin(dLng / 2) * Math.sin(dLng / 2);
        double c = 2 * Math.atan2(Math.sqrt(a), Math.sqrt(1 - a));
        return EARTH_RADIUS_METERS * c;
    }
}
```

설명: 지구를 반지름 6,371km 구로 근사하여 두 좌표의 대원거리(great-circle distance)를 계산한다. 외부 지도 API 호출 없이 서버 내 연산만으로 ms 단위 판정이 가능해, 인증 요청이 몰려도 외부 의존성 병목이 없다.

| 핵심 소스코드 (2) — 일일 GPS 인증 처리(핵심 비즈니스 흐름)

위치 확인 → 중복 방지 → 반경 판정 → 스트릭/출석/코인 보상까지 하나의 트랜잭션으로 처리.

```

// personalcheckin/service/PersonalCheckInService.java
@Transactional
public CheckInResponse checkIn(Long userId, double currentLat, double currentLng) {
    LocalDate today = LocalDate.now(clock);

    // User 를 비관적 락으로 '먼저' 로드 → 코인·출석·스트릭 갱신을 사용자 단위로 직렬화하고
    // active 를 락 상태에서 확인(TOCTOU 차단). 복귀 수행과 락 순서 일치 → 데드락 없음.
    User user = userRepository.findByIdForUpdate(userId)
        .orElseThrow(() -> BusinessException.notFound("USER_NOT_FOUND", "사용자를 찾을 수 없습니
            다."));
    if (!user.isActive()) {
        throw BusinessException.forbidden("INACTIVE_USER", "비활성(탈퇴) 계정입니다.");
    }

    // (F2 정책) 오늘이 복귀 대상일(오늘 마감 복귀 미션 존재)이면 일반 인증 차단 -
    // 복귀 수행이 곧 그날의 인증이다(인증 카운트·순서 의존 방지).
    OffsetDateTime dayStart = today.atStartOfDay(clock.getZone()).toOffsetDateTime();
    OffsetDateTime dayEnd = today.atTime(23, 59, 59).atZone(clock.getZone()).toOffsetDateTime();
    if (recoveryMissionRepository.existsByUserIdAndDeadlineAtBetween(userId, dayStart, dayEnd)) {
        throw BusinessException.conflict("RECOVERY_DAY_NO_CHECKIN",
            "복귀 미션 대상일에는 복귀로 인증됩니다. 별도 일반 인증은 불가합니다.");
    }

    PersonalLocation location = personalLocationRepository.findById(userId)
        .orElseThrow(() -> BusinessException.badRequest(
            "LOCATION_NOT_REGISTERED", "개인 GPS 위치를 먼저 등록하세요."));

    // 당일 중복 인증 방지 (이미 SUCCESS면 409)
    PersonalCheckIn existing = personalCheckInRepository
        .findByUserIdAndDate(userId, today).orElse(null);
    if (existing != null && existing.isSuccess()) {
        throw BusinessException.conflict("DUPLICATE_CHECK_IN", "오늘 이미 인증을 완료했습니다.");
    }

    // GPS 반경 판정 - 실패 시 레코드 생성하지 않음
    boolean within = gpsEvaluator.isWithinRadius(
        location.getLat(), location.getLng(), location.getRadiusMeters(),
        currentLat, currentLng);
    if (!within) {
        throw BusinessException.badRequest("GPS_OUT_OF_RANGE", "등록된 위치 반경을 벗어났습니다.");
    }

    OffsetDateTime now = OffsetDateTime.now(clock);
    if (existing != null) {
        existing.markSuccess(now); // 복귀 대기 레코드 보정
    } else {
        personalCheckInRepository.save(PersonalCheckIn.success(userId, today, now));
    }

    user.increaseAttendance(); // 출석 +1

    Streak streak = streakRepository.findById(userId).orElseThrow(...);
    streak.recordSuccess(today); // 스트릭 +1(갯이면 1로 재시작)

    boolean rewardGranted = false;
    if (isRewardEligible(streak.getCurrentStreak())) { // 마일스톤 도달 시
        coinService.grant(userId, rewardCoins, CoinTransactionReason.STREAK_REWARD, null);
        rewardGranted = true;
    }
}

```

```

    }
    return new CheckInResponse(today, SUCCESS, now,
        streak.getCurrentStreak(), streak.getMaxStreak(), user.getCoinBalance(), rewardGranted);
}

```

설명: User 행을 비관적 락으로 먼저 잡아 동시 인증·복귀를 사용자 단위로 직렬화하고 활성 계정을 락 상태에서 재확인한다. 오늘이 복귀 대상일이면 일반 인증을 막아(RECOVERY_DAY_NO_CHECKIN) 복귀 수행이 곧 그날의 인증이 되게 한다. `UNIQUE(user_id, check_in_date)` 제약 + 애플리케이션 단 중복 검사로 하루 1회 인증을 보장하고, GPS 실패 시에는 레코드를 만들지 않아 데이터가 오염되지 않으며, 인증·출석·스트릭·코인 변경이 하나의 `@Transactional` 로 묶여 부분 반영을 방지한다.

| 핵심 소스코드 (3) — 코인 단일 진실 원천(Single Source of Truth)

모든 코인 변동을 한 메서드로만 처리해 잔액 = 거래내역 합계 불변식을 보장한다.

```

// coin/service/CoinService.java
@Transactional
public long charge(Long userId, long amount, CoinTransactionReason reason, Long referenceId) {
    if (amount < 0) throw new IllegalArgumentException("charge amount must be >= 0: " + amount);
    User user = lockUser(userId); // 비관적 락 (동시 차감 방지)
    long effective = Math.min(amount, user.getCoinBalance()); // 잔액까지만 차감(음수 방지)
    user.addCoins(-effective);
    coinTransactionRepository.save(
        CoinTransaction.of(userId, reason, -effective, user.getCoinBalance(), referenceId));
    return effective; // 실제 차감액 반환
}

private User lockUser(Long userId) {
    return userRepository.findByIdForUpdate(userId) // SELECT ... FOR UPDATE
        .orElseThrow(() -> BusinessException.notFound("USER_NOT_FOUND", "사용자를 찾을 수 없습니
            다."));
}

```

설명: 지급(`grant`)·차감(`charge`)이 항상 `CoinTransaction` 기록과 `User.coinBalance` 갱신을 함께 수행하므로 `SUM(amount) == coinBalance` 가 항상 성립한다. `SELECT ... FOR UPDATE` 비관적 락으로 동시 요청에도 잔액 정합성이 깨지지 않으며, 부하 테스트(BS-30)에서 동시성 오류 0건으로 검증되었다.

| 핵심 소스코드 (4) — 복귀 성공 처리(복귀일 = 그날의 인증으로 간주)

놓친 날을 보정하면서, 복귀한 날 자체를 그날의 인증으로 간주해 출석·스트릭을 +1 한다(F2 팀 결정).

```

// recovery/service/RecoveryService.java (핵심 발췌)
@Transactional
public RecoveryResultResponse performRecovery(Long userId, double lat, double lng) {
    // PENDING 미션을 비관락으로 획득 → 동시 수행 시 정확히 1회만 성공
    RecoveryMission mission = recoveryMissionRepository
        .findPendingByUserForUpdate(userId).stream().findFirst()
        .orElseThrow(() -> BusinessException.notFound("NO_RECOVERY_MISSION", "대기 중인 복귀 미션이 없습니다."));

    // 마감 이내(현재 < deadline)만 성공 - 경계에서 스케줄러 실패(-100)와 이중차감 방지
    OffsetDateTime now = OffsetDateTime.now(clock);
    if (!now.isBefore(mission.getDeadlineAt())) {
        throw BusinessException.badRequest("RECOVERY_EXPIRED", "복귀 미션 데드라인이 지났습니다.");
    }
    // ... GPS 변경 판정(반경 밖이면 GPS_OUT_OF_RANGE), User 비관락 + active 재확인 ...

    mission.complete(now);
    PersonalCheckIn missed =
        personalCheckInRepository.findById(mission.getPersonalCheckInId()).orElseThrow(...);
    missed.markSuccess(now); // 미인증일 → SUCCESS 보정

    // (F2·F8) 복귀 = 복귀한 날의 인증으로 완전 간주. 단, 스케줄러 생성 전(00:00~00:01) 창에서
    // 오늘 일반 인증이 먼저 들어왔다면 오늘 크레딧(레코드·출석·스트릭)은 중복 지급하지 않는다.
    LocalDate today = LocalDate.now(clock);
    boolean todayAlreadyDone = personalCheckInRepository.findByUserIdAndDate(userId, today)
        .map(PersonalCheckIn::isSuccess).orElse(false);
    if (!todayAlreadyDone) {
        personalCheckInRepository.save(PersonalCheckIn.success(userId, today, now)); // 오늘 SUCCESS 레코드
    }

    coinService.charge(userId, successPenalty, RECOVERY_SUCCESS, mission.getId()); // -50(클래핑) - 항상 부과

    Streak streak = streakRepository.findById(userId).orElseThrow(...);
    if (!todayAlreadyDone) {
        user.increaseAttendance(); // 출석 +1
        streak.recordRecoverySuccess(today); // 복귀일을 인증으로 간주해 스트릭 +1 (마일스톤 보상 없음)
    }
    return new RecoveryResultResponse(mission.getId(), mission.getStatus(),
        mission.getCompletedAt(), streak.getCurrentStreak(), user.getCoinBalance(), charged);
}

```

설명: 복귀 페널티(-50)는 항상 부과하고, 복귀일을 그날의 인증으로 간주해 **출석·스트릭을 +1** 한다(단 일일 인증과 달리 %주기 마일스톤 보상 코인은 지급하지 않음 — 팀 결정). 놓친 날과 오늘을 모두 SUCCESS로 보정해 복귀일이 다음 날 다시 미인증으로 잡히는 무한 복귀 루프를 막는다. PENDING 미션을 **FOR UPDATE** 비관락으로 잡아 동시 수행에도 정확히 1회만 성공하며, "현재 < 마감" strict 경계로 스케줄러 실패 처리(-100)와의 이중 차감을 방지한다.

(참조) 데이터 모델 — A축 테이블

테이블	PK	주요 컬럼	관계/제약
users	id	email(unique), password_hash, nickname, coin_balance, total_attendance, is_active, joined_at	모든 A축 도메인의 기준
personal_locations	user_id	lat, lng, radius_meters, place_name	user_id 공유(1:1)
streaks	user_id	current_streak, max_streak, last_success_date	user_id 공유(1:1)
personal_check_ins	id	user_id, check_in_date, status, verified_at	UNIQUE(user_id, check_in_date)
recovery_missions	id	personal_check_in_id(unique), user_id, deadline_at, completed_at, status	check_in과 1:1
coin_transactions	id	user_id, type, amount(부호 있음), balance_after, reference_id	코인 변동 단일 이력

※ 연관관계는 JPA 객체 매핑이 아니라 **논리 FK(Long id)**로 설계 → 의도적으로 N+1 쿼리 0건 (BS-30 검증). ※ status enum: PersonalCheckIn = **SUCCESS / RECOVERY_PENDING / FAILED** , Recovery = **PENDING / COMPLETED / FAILED** .